uVote - Luke Individual Contribution Report

Name: Luke Doyle

Student ID: D00255656

Introduction

This document presents my individual contribution to uVote, a secure online voting system for small-scale elections.

uVote runs as six FastAPI microservices on a Kubernetes (Kind) cluster, with PostgreSQL, zero-trust Calico network policies, an ELK logging stack, and Prometheus metrics.
My scope covered the Kubernetes platform, the database and its security model, the observability stack, test architecture, and the architectural and process documentation.

The document is organised into three sections.
Section 1 to 5 covers the implementation work I personally carried out.
Section 6 documents my use of Jira and Kanban for tracking work.
Section 7 is a critical reflection on what went well, what did not, and what I learned.

1. Kubernetes Platform and Network Policies

1.1 Cluster Setup and Configuration

The Kind cluster is defined in `kind-config.yaml`.
The Cluster is made from a control-plane and worker node with port 80 and 443 mapped to the control-plane container which handles ingress.
A subnet for the pods is configured to be 10.244.0.0/16, the same as defined in `custom-resources.yaml` in Calico.

The `setup_k8s_platform.py` automates cluster creation from scratch. The method `create_kind_cluster()` runs the following command;

```
kind create cluster --config kind-config.yaml --name uvote
```

This creates the cluster with the name uvote.
The script then manually applies the label `ingress-ready=true` to the control-plane node. Kind automatically labels the node when the cluster name is the default kind. If it doesn't have the label, the ingress controller's nodeSelector cannot schedule on the control-plane and the ingress never becomes reachable.
This script is easier then using `kubectl` commands as it reduces the toil of bringing up and down the cluster during development as my laptop can not handle multiple clusters at once. My laptop nearly died trying to run the full cluster at one point.

The first stage of `deploy_platform.py` executes pre-deployment checks before any deployment work begins, including checking for the cluster name and the existence of the “uvote-dev” namespace. If the namespace is missing, the script ends and logs the output.

1.2 Platform Security

We selected Calico for initial security and later added Istio on top of Calico after our Microservices CA.

`kind-config.yaml` specifies `disableDefaultCNI: true` to prevent the installation of kindnet. No enforcement of NetworkPolicy objects by kindnet despite being available in Kubernetes. Policies are permitted to be applied without error, but they are ignored silently. It was decided to replace it with Calico, since a CNI that does not enforce NetworkPolicy achieves zero-trust model.

However, a CNI alone does not meet zero-trust. It does not matter what request types can be sent; only which pods can interact and which cannot. A service mesh is necessary for that.

In `setup_k8s_platform.py`, the function `install_calico()` triggers the installation of Calico in two steps.

Start with the Tigera operator, then the Installation and APIServer custom resources from a local copy of custom-resources.yaml. The file sets up the encapsulation and locks the to 10.244.0.0/16
To remove the network dependency during cluster setup, a local copy is used rather than the upstream URL to pin the CIDR at a known value.

Istio operates in sidecar configuration. An Envoy proxy has been implanted into each application pod, directing all inbound and outbound traffic through the proxy for that pod. The mesh implements mTLS, authorization policies, retries, timeouts, circuit breaking, and distributed tracing without requiring any modifications to application code.

Calico and Istio work together at different layers of the system. Think of Calico as "who can talk to who" and Istio as "what can be said." For any connection to succeed, it must satisfy both.

Calico operates primarily at Layer 3 and Layer 4, defining which specific pods can initiate a network connection depending on their IP address and port number. If Calico blocks a connection, the packet won't reach.
Istio operates at Layers 4 and 7. It regulates the permitted communications over a connection that Calico has already allowed. It enforces mTLS identity, HTTP-level permission frameworks, and traffic management regulations such as weighted routing and fault injection.

By using various layers, we utilize the zero-trust model at the network and the application levels.

1.3 Namespace Strategy

All six application services and PostgreSQL run in uvote-dev. The full suite of monitoring components runs in a single, dedicated monitoring namespace; Elasticsearch, Kibana, FluentBit, Prometheus, Grafana.
By seperating the services, the monitoring workloads are unable to target the application's label selectors; hence, ELK stack resource limits do not affect application pod scheduling.

`boot_platform.py` expressly sequences the two deployments;

1. K8s cluster and namespaces creation
2. Executing `deploy_platform.py` for the uvote-dev services
3. Helm installs ELK monitoring

1.4 Zero-Trust Network Policy Design and Implementation

Ten YAML files in `uvote-platform/k8s/network-policies/` implement a default-deny-all model.
The `deploy_platform.py` function `apply_network_policies()` applies them in order of the file numbers (00 → N).

File	Purpose
`00-default-deny.yaml`	Block all traffic by default
`01-allow-dns.yaml`	Egress to CoreDNS on UDP/TCP 53
`02-allow-to-database.yaml`	Six whitelisted services to PostgreSQL on 5432
`03-allow-from-ingress.yaml`	Ingress controller to each exposed service
`04-allow-audit.yaml`	Services to audit-service, bidirectional
`05-allow-mailhog.yaml`	admin-service to MailHog on 1025 and 8025
`06-allow-prometheus-scrape.yaml`	Prometheus to each service's `/metrics` endpoint
`07-allow-prometheus-egress.yaml`	Prometheus egress to all six services

`08-allow-inter-service.yaml`	Cross-service HTTP calls between specific pairs
`09-allow-grafana-prometheus.yaml`	Grafana to Prometheus on 9090

The policy file, `02-allow-to-database.yaml` uses `matchExpressions` with `operator: In` for the selection of six backend services in one podSelector block only. This is policy dictates what pod an access the database.
The Frontend Service is not included in that list as it has no reason to.

2. Database Design and Security

2.1 PostgreSQL Deployment

In `k8s/database/db-deployment.yaml`, PostgreSQL is deployed in a single-replica Deployment using the `postgres:15-alpine` image. The database gets set up using a `ConfigMap`, which mounts multiple files into the path `/docker-entrypoint-initdb.d/`. These files include `schema.sql`, `create_roles.sql`, and `seed_data.sql`.

`uvote_admin` is superuser and database is called `uvote`. These are set via the `POSTGRES_USER` and `POSTGRES_DB` environment variables.

The article asserts that persistent storage is a `PersistentVolumeClaim` of 5Gi with `ReadWriteOnce` access backed by the standard storage class of Kind. I chose 5Gi just so we have more then enough space for testing and basic demoing. The real system should require alot more.

2.2 Schema Design and Anonymity Model

The anonymity model is enforced at the data level. This schema comment block states it directly:

```
-- Design principle: ANONYMITY vs ACCOUNTABILITY
--   We can prove a voter voted (prevent double-voting)
--   We CANNOT determine how they voted (ballot secrecy)
--   The encrypted_ballots table has NO voter_id / user_id foreign key
--   Linkage between identity and choice is broken by blind ballot
tokens

```

The “anonymity break” happens across two tables.
A `voting_token` connects voters to an election. Voter clicks email link, this token checks out and sets `voters.has_voted = true`.

At that point, the voter's identity has been verified. Blind tokens lack both voter and voting token IDs. Once the MFA is completed, 1auth-service1 assigns the voter a blind token, severing their identity from their ballot authorization.

`encrypted_ballots` has no `voter_id`, no `user_id`, and no `ballot_token_id`.
Once a blind token is consumed, the ballot record contains only `election_id`, `encrypted_vote`, `ballot_hash`, `previous_hash`, `receipt_token`, and `cast_at`.
Even with full database access, no one knows who voted for who, just that they voted.
This is documented in ADR-015.

2.3 Per-Service Database Roles

`create_roles.sql` creates six PostgreSQL users, each with only the grants their service needs.

Role	Permissions
`auth_service`	SELECT/INSERT/UPDATE on organisers; SELECT/UPDATE on voters; SELECT/INSERT on voter_mfa; INSERT on blind_tokens
`voting_service`	INSERT on ballot and receipt tables; SELECT on election data; SELECT/UPDATE on tokens
`election_service`	Full CRUD on elections and election_options
`results_service`	SELECT on ballot and election tables; SELECT/INSERT/UPDATE on tallied_votes
`audit_service`	INSERT and SELECT on audit_log only
`admin_service`	SELECT/INSERT/UPDATE/DELETE on voter management tables

Passwords are placeholders in the SQL file.
When deploying with `deploy_platform.py`, the script generates random secrets and patches them into the `db-credentials` Kubernetes Secret. This follows a concept I heard of “If I don’t know, you don’t know”. This can give an additional layer of trust that I can add, alter or

remove votes.
This is documented in ADR-014.

The network policy `02-allow-to-database.yaml` enforces the same access list at the network layer. Only the six service pods can reach port 5432.
Even if an attacker gains a pod with an allowed label, they still need valid database credentials to be able to get into the Database. And if those unlikely cases, the Hash Chained Logs provide additional security.

2.4 Ballot Immutability

Two PostgreSQL triggers enforce immutability at the database level, preventing any `UPDATE` or `DELETE` on the ballot and audit tables regardless of which role attempts it.

```
CREATE OR REPLACE FUNCTION prevent_ballot_modification()
RETURNS TRIGGER AS $$
BEGIN
    RAISE EXCEPTION 'Encrypted ballots are immutable and cannot be
modified or deleted';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER immutable_ballots
    BEFORE UPDATE OR DELETE ON encrypted_ballots
    FOR EACH ROW EXECUTE FUNCTION prevent_ballot_modification();

```

A matching pair, `prevent_audit_modification` and `immutable_audit`, protects `audit_log`. These triggers fire at the PostgreSQL level.
A compromised service with full database credentials still cannot alter the ledger.

2.5 Audit Log Hash Chain

Two `BEFORE INSERT` triggers auto-generate SHA-256 hashes for both `encrypted_ballots` and `audit_log` using pgcrypto.

```
NEW.ballot_hash := encode(
    digest(
        NEW.election_id::text || NEW.encrypted_vote::text ||
        NEW.cast_at::text || gen_random_uuid()::text,
        'sha256'
    ),
    'hex'
);

```

The salts generated by `gen_random_uuid()` ensure that identical votes produced by the same election at the same timestamp have different hashes. The `previous_hash` column in both tables attaches each row to the one before it, establishing a link chain.

Any change to a historic record breaks the chain, as such the results page checks the full chain on every page load.
This is documented in ADR-007.

2.6 Schema Divergence and Resolution

After Stage 1, the `database/init.sql` diverged from `k8s/database/schema.sql`. This platform schema still references table names such as admins, candidates, audit_logs that are from an older version.
I merged the 2 schema together to create what we currently use. As a result, `UndefinedTableError` crashes occurred when services were started against the platform database.

The `schema.sql` file was completely over-written using `init.sql` as the canonical source. It also moved the incorrect table references into `create_roles.sql` and `seed_data.sql`. A change was made in `deploy_platform.py` to apply the three files in strict order.

1. schema.sql
2. create_roles.sql
3. seed_data.sql

Thusly halting on any failure.

Through a hash chaining library, a unique link was added to each output event by the outbox poller. The triggers were documented in the documentation. They were not in running the database. After more effort, they became best of friends.

---

3. Deployment and Boot Orchestration

3.1 Build and Deploy Pipeline

The `deploy_platform.py` has the PlatformDeployer class.
It executes a sequence of ten phases.

Phase 2 generates the Docker images for each service, and each is re-tagged as `latest` to ensure alignment between Kind's image load and the deployment YAML references.
Each image is loaded into Kind using kind load docker-image.
Phase 5 applies the deployment manifests backend first to frontend last from `/k8s/services/`.

All service metadata (deploy name, manifest filename, container port, health path, database access) is centralised together in a `SERVICE_REGISTRY`.
All phases iterate over a common list, providing one single source of truth. That list eliminates duplication and guarantees no service gets overlooked.

Utilizing a `--skip-build` flag enables Phase 2 to confirm the local existence of images through docker image inspect as opposed to rebuilding them.
The outcome will be efficiency of re-deploys when the code has not changed.

3.2 Secret Management and Password Sync

Phase 4 creates or patches four Kubernetes Secrets: `db-credentials`, `jwt-secret`, `flask-secret`, and `smtp-credentials`.

Both the password key used by the other services as `DB_PASSWORD` and the `postgres-password` key used by the PostgreSQL container as `POSTGRES_PASSWORD` is provided by db-credentials.
Because `uvote_admin` is the superuser of PostgreSQL, both must be identical.
The deploy script verifies a secret already exists, checks for divergence between the two keys, and patches them as necessary.

The command `_sync_pg_password()` then issues an `ALTER USER uvote_admin PASSWORD` in SQL in the PostgreSQL pod via kubectl exec.
No matter if the secret was just created, patched, or left unchanged, it still runs, so the in-database password always matches the secret, even given a different one at cluster init.

The values for `uvote-dev` in `smtp-credentials` are prefilled with MailHog values. The admin-service deployment reads SMTP configurations from `secretKeyRef`, making the manifest environment-agnostic.

3.3 Health Checks and Pre-Flight Verification

Phase 7 polls `kubectl get pods -n uvote-dev -o json` in a loop, checking that all pods are in phase Running and all container statuses are `ready: true`.
Without bursting it immediately as some pods may recover, CrashLoopBackOff; ImagePullBackOff are detected and logged.
Upon reaching the timeout, each non-ready pod will have their last 30 lines of logs captured via kubectl previous. It could be argued to have more then 30 lines, but this was to save on storage space in the files.

The function `_exec_tcp_check` tests connectivity to the Host pod through the pods Port.
If `nc` is not available, it default to `bash -c '</dev/tcp/<host>/<port>'`.
This works no matter what tools the base image comes with.

The function `_health_via_port_forward()` creates a subprocess to the `kubectl port-forward` that uses a random free local port, then polls until the tunnel accepts connections, then makes an HTTP GET against the health endpoint using `urllib.request`.
There is no dependency on curl or any tool inside the container, this was an experiment that worked out but I am more used to using curl.

The function `_resolve_pod_name()` uses `-l app=<name>,tier=backend` to find the pod for the service. The need for the `tier=backend` filter exists due to `test-pods.yaml` deploying a test pod with label `app=auth-service` to test network policies.
If the filter is not applied, the execution of `kubectl exec` could land on the test pod that runs `sleep infinitely`.

3.4 Single Command Boot Orchestration

`boot_platform.py` brings the entire platform from zero to running in one command.

Step	Script	What it does
1	`setup_k8s_platform.py`	Cluster, CNI, DB, namespaces
2	`deploy_platform.py`	Build, load, secrets, services, policies, health
3	ELK stack	Helm install Elasticsearch, Kibana, Fluent Bit
4	Final health check	All pods in uvote-dev and monitoring
5	`create_dashboard.py`	Kibana dashboard via port-forward
6	K8s dashboard	Bearer token reminder

The `boot_test_platform.py` serves the same purpose for the test environment, using `deploy_test_platform.py`, which targets the `uvote-test` namespace and adds MailHog.

Mailhog is used for testing the OTP email.

3.5 Scripting Language Choice

All platform scripts are written in Python. This was because we, Hafsa and I, both learned to program in Python so we are familiar with the language, and so the scripts run regardless of the operating system.

4. Observability

4.1 Structured Logging

Prior to this, most services had no effective logging.Each service either had no logger or a unique one per serice.
Debugging issues across the cluster requires reading the raw output from containers as there is no fixed format.

The six services all call `configure_logging()` at startup from `shared/logging_config.py`.

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'json': {
            'format': '%(asctime)s %(name)s %(levelname)s %
(message)s',
            'datefmt': '%Y-%m-%dT%H:%M:%SZ',
            'class': 'pythonjsonlogger.jsonlogger.JsonFormatter',
        }
    },
    'handlers': {
        'stdout': {
            'class': 'logging.StreamHandler',
            'stream': 'ext://sys.stdout',
            'formatter': 'json',
        }
    },
}

```

Each service creates a named logger, for example `logging.getLogger('auth-service')`. The Kibana service can group logs by the service without the need for the Kubernetes pod name thanks to the `name` field in the JSON output. The logs are directed to the container's standard output, which is monitored by Fluent Bit. The structured fields consist of `asctime`, `name`, `levelname`, `message`.

4.2 Log Aggregation Stack

Step 3 of `boot_platform.py` deploys Three Helm charts the `monitoring` namespace.

Elasticsearch runs as a single-node instance. `discoveryType: "single-node"` is set at the chart level rather than in `esConfig`. Memory is capped at 2Gi to avoid starving application pods.

Fluent Bit runs as a DaemonSet. It tails `/var/log/containers/*uvote-dev*.log` to capture only the application namespace.

Kibana is resource-capped at 2Gi and 1000m CPU. The `create_kibana_service_token()` method in `deploy_platform.py` manages the service account token lifecycle: it deletes any existing token, creates a fresh one inside the `elasticsearch-master-0` pod, parses the token from the output line, and stores it in a `kibana-service-account-token` Secret in the `monitoring` namespace using the `--dry-run=client | kubectl apply` idempotent pattern.

4.3 Log Dashboard Provisioning

`plat_scripts/create_dashboard.py` provisions a Kibana dashboard automatically via the Kibana saved-objects bulk API and runs after a port-forward to port 5601.
It uses stable UUIDs for all objects so the dashboard survives a cluster restart or reprovisioning without duplicate creation.

The data view will `uvote-logs*` with `@timestamp` as the time field, creates 6 visualisations and 1 dashboard object.
The visualizations include number of errors over time, auth failure rate, vote submission rate, email failure rate, top messages and live error streaming from all services.

Step 5 in `boot_platform.py` uses `kubectl` to extract the Elasticsearch password from the cluster Secret. Then it port-forwards Kibana to 5601 before setting ES_PASSWORD in the environment.
Finally, it executes the script in the subprocess. You can find the dashboard on every fresh cluster without any manual steps.

5. Test Architecture

5.1 Database Integration Tests

`tests/test_db.py` is a live-cluster integration test suite that runs directly against PostgreSQL through kubectl exec into the database pod.
It doesn't mock or fake anything.
Each assertion is executed with respect to the actual schema in the running cluster.

The suite has 10 tests covering:

- All 12 tables exist: `organisations`, `organisers`, `elections`, `voters`, `voter_mfa`, `election_options`, `voting_tokens`, `blind_tokens`, `encrypted_ballots`, `vote_receipts`, `tallied_votes`, `audit_log`
- The `immutable_ballots` trigger exists on `encrypted_ballots` and `immutable_audit` exists on `audit_log`, verified via `information_schema.triggers`. A live-fire `UPDATE` is explicitly skipped because inserting into `encrypted_ballots` requires `pgp_sym_encrypt` with a valid encryption key, which is not available in the test environment
- The `auto_ballot_hash` trigger exists on `encrypted_ballots`, also via catalogue query
- Per-service role permissions: `auth_service` can `SELECT` on `organisers`, `results_service` can `SELECT` on `encrypted_ballots` and cannot `INSERT`

The test runs in two modes. Running `tests/test_db.py` with quick flag gives plain output. Running `pytest tests/test_db.py` resolves the pod name with `kubectl get pods -l app=postgresql` through ten wrapper functions. The hook `pytest_collection_modifyitems` inside `conftest.py` filters the standalone function names from pytest discovery so only the wrapper functions are collected.

Once the schema divergence was resolved, the test assertions were up to date with the current table names. The previous versions were making claims about `admins`, `candidates`, and `votes`, which no longer exist.

5.2 Shared Test Fixtures and Email Testing

`tests/conftest.py` provides session-scoped fixtures shared across all integration tests.

The `mailhog` fixture initiates a subprocess for `kubectl port forwarding svc/mailhog 8025:8025 using kubectl`, polls for TCP connectivity at 0.5 second intervals up to a 10 second maximum. It yields a `MailHogHelper` instance. It tears down the port-forward on the session end as above. A graceful terminate is attempted, and if that times out, a SIGKILL is sent.

The `clear_mailhog` fixture runs automatically before every test without an explicit call. It clears the MailHog inbox through `asyncio.run()` wrapped in a try/except to ensure a MailHog failure never blocks a test from running. The `UVOTE_NAMESPACE` environment variable provides the namespace instead of a hardcoded one, with defaulting to `uvote-dev`.

The function `get_latest_voting_token(voter_email)` found in `tests/helpers/mailhog.py` queries `localhost:8025/api/v2/messages` to match the recipient and then pull the voting token from the `/vote/{token}` pattern of the email.

5.3 Shared Module Unit Tests

The Python test located in `tests/test_shared.py` independently covers `shared/security.py`, `shared/schemas.py`, and `shared/database.py` without any reliance on a running service or cluster.

The tests cover:

- `token_urlsafe(32)` produces a 43-character URL-safe string
- `BALLOT_TOKEN_SECRET` does not fall back to a hardcoded default when the environment variable is set, which would be a security risk
- bcrypt round-trip: `hash_password()` produces a hash, `verify_password()` returns `True` for correct input and `False` for wrong
- JWT payload round-trip with `organiser_id`, `email`, and `exp` fields intact
- Pydantic models raise `ValidationError` on missing required fields and reject invalid email addresses
- `TokenVerifyResponse` does not leak `election_id` or `voter_id` in its output
- `asyncpg.create_pool` is called with the correct `DATABASE_URL` environment variable
- `get_db()` raises a meaningful error if the pool is uninitialised

The `tests/test_logging_config.py` file verifies the output formatting and logger name correctness of the `configure_logging()` function.

5.4 In-Cluster Email Testing Infrastructure

The email testing was difficult to figure out. I mentioned this issue to my lectuer, Michelle, and she gave me the idea for in cluster SMTP. I then found MailHog.

In `k8s/mailhog/mailhog-deployment.yaml`, MailHog is set up to be an internal fake SMTP server. Access to the SMTP protocol is available on port 1025 for the mail system, and HTTP API on port 8025 as a ClusterIP service.

The `05-allow-mailhog.yaml` network policy satisfies the “bidirectional pattern”.
The egress permission for MailHog is enabled by allow-mailhog-egress on port 1025 from app:admin-service.
The ingress permission for MailHog is enabled by `allow-mailhog-ingress` on port 1025, for Traffic from `app:admin-service`. The deployment of the admin service is read by `secretKeyRef` SMTP from the `smtp-credentials`. This creates a compatibility manifest.
The deploy script establishes the secret containing the MailHog values for uvote-dev and makes clear its dev-only status in the comments.

This replaced an INSERT workaround using kubectl exec psql that was created during the original `test_api.py` bypassing email delivery completely. The new flow makes an actual token generation request using JWT authentication, it polls MailHog for the email, extracts the token from the URL and clears the inbox between tests.

---

6. Documentation

6.1 Architecture Decision Records

Fifteen ADRs were authored covering all major design choices. Each follows the EdgeX ADR template: Status, Context, Options Considered with pros and cons, Decision with rationale, Consequences, Implementation Notes, and Validation criteria.
I wrote these using AI assistance after the team change in the start of Semester 2 to lay the ground work of what was to come. Less then ideal.

ADR	Title	Category
ADR-001	Python FastAPI Backend	Backend Framework
ADR-002	PostgreSQL Database	Database
ADR-003	Kubernetes Platform	Infrastructure
ADR-004	Calico CNI Networking	Networking
ADR-005	Token-Based Voter Authentication	Authentication
ADR-006	JWT Admin Authentication	Authentication
ADR-007	SHA-256 Hash-Chain Audit Logs	Security
ADR-008	Microservices Architecture	Architecture
ADR-009	Server-Side Rendering (Jinja2)	Frontend
ADR-010	Zero-Trust Network Policies	Security
ADR-011	Kubernetes Secrets Management	Security

ADR-012	Kind for Local Development	Infrastructure
ADR-013	Domain-Driven Service Separation	Architecture
ADR-014	Per-Service Database Users	Database
ADR-015	Blind Ballot Token Anonymity	Security

Some of the ADR decisions have been changed, such as Calico vs Istio. I wrote that without fully understanding how they can work together, which i only learned after using them together in my Microservices Class.

6.2 Project Documentation

`setup_venv.py` is a script that creates the project virtual enviornment.
It supports `--clean`, `--verify-only`, and `--help` flags. It was written after a missing `colorama` dependency was found to be blocking `deploy_platform.py` from running on fresh checkouts.

`.docs/SETUP.md` is the developer quickstart covering venv activation, common test commands, and the full boot sequence.

The Stage 1 platform report, network security document, build log, and the Stage 2 poster and presentation are also mine.
The Stage 2 poster was produced in Flowchart Maker & Online Diagram Software and includes the C4 service diagram, and the voter flow diagram, all of which I drew. It also includes the CI pipeline diagram, which Hafsa drew.

7. Reflection

This project was genuinely difficult, and most of that difficulty came down to time.

In Semester 1 we had a conflict within the original team that Andrew resolved by splitting the group. Hafsa and I started from nothing in Semester 2 while other teams were already well ahead. That was the backdrop for everything that followed: every late night, every gap between what was documented and what was running.
We got to a platform and a working application by Stage 1, and while the Kubernetes

deployment was not fully demonstrated at that point, we had gone from nothing to roughly on par with the other teams.
That did not come without cost, but the cost was mostly sleep and a new Red Bull addiction rather than our other assignments.

What I take away technically is an understanding of security that has genuinely changed how I think. My go-to question in class became "how does this impact security?" to the point where Andrew suggested it might be my niche.
That has carried into my personal life too, to the point of installing a custom OS on my phone. That shift did not come from just reading about security, but from building a system where the wrong schema design means a voter can be identified, where a missing network policy means services that should never communicate can, where a trigger that does not exist means an audit log that claims to be immutable is not. It was a headache, but a worthwhile one looking back on it.

An interesting takeaway is, I have more respect for graphic designers. Creating the poser was hours of me hunched over trying to get pixel to align and trying to get enough detail into the poster. I had to ask my uncle who has a degree in Art and works in Tech for advice on how to lay it out. So I think I am good with never touching Graphic design again.

My biggest regret is the Semester 1 conflict, and the timing of the most valuable classes. Software Design and Microservices both ran in Semester 2. The patterns I learned there, SAGA, CQRS, Transactional Outbox, Ports and Adapters, proper aggregate boundaries, are exactly what uVote needed from the start. Without the conflict, and with that knowledge going in, uVote would have been a different system entirely. Part of me wants to go back and build that version just to see what it could actually be.

Despite everything, this is the project I am most proud of.

8. Use of AI

AI was used throughout the code aspect of this project as per the project spec.

AI was used to help summerise information from the project to help me write this individual report.
This report was not generated by AI, everything written I typed myself.